

III B.Tech I Semester

R233PC52	NATURAL LANGUAGE PROCESSING (Professional Core)	L	T	P	C
		3	0	0	3

Course Objectives

- Basics of NLP, Morphology, Tokenization, N-gram Models
- POS Tagging, Parsing, Treebanks, Ambiguity Handling
- Word Sense Disambiguation, Semantic Parsing, Sentiment Analysis
- Machine Translation, Transformers, BERT/GPT, Ethical NLP
- Speech Recognition, Feature Extraction, Discourse Analysis

Course Outcomes

- Understand morphological processing and the structure of words and documents.
- Analyze syntactic structures using various parsing algorithms.
- Apply semantic parsing techniques to interpret natural language text.
- Understand predicate-argument structures and meaning representation systems.
- Apply cross-lingual language models and speech recognition techniques in NLP applications

UNIT I: Introduction to NLP

lectures: 9 hours

Introduction to NLP: Origins and Challenges, Language and Grammar in NLP, Regular Expressions and Finite-State Automata, Tokenization: Text Segmentation and Sentence Splitting, Morphological Parsing: Stemming and Lemmatization, Spelling Error Detection and Correction, Minimum Edit Distance and Applications, Statistical Language Models: Unigram, Bigram, and Trigram Models.

UNIT II: Word-Level and Syntactic Analysis

lectures: 9 hours

Introduction, Part-of-Speech (POS) Tagging: Rule-Based, Stochastic and Transformation-Based Approaches, Hidden Markov Models (HMM) and Maximum Entropy Models for POS Tagging, Context-Free Grammar (CFG) and Constituency Parsing, Treebanks and Normal Forms for Grammar, Top-Down and Bottom-Up Parsing Strategies, CYK Parsing Algorithm.

UNIT III: Text Classification and Information Retrieval

lectures: 9 hours

Naïve Bayes Classifier for Text Classification, Training and Optimization for Sentiment Analysis, Information Retrieval: Basic Concepts and Design Features, Information Retrieval Models: Classical, Non-Classical, Cluster Model, Fuzzy Model, **Word Sense Disambiguation (WSD) Methods: Supervised and Dictionary-Based Approaches, Introduction to Sequence Labelling.**

UNIT IV: Machine Translation and Semantic Processing

lectures: 11 hours

Introduction to Machine Translation (MT), Language Divergence and Typology in MT Encoder- Decoder Model for Machine Translation, Translating in Low-Resource Scenarios, MT Evaluation Metrics and Techniques, Semantic Analysis and First-Order Logic in NLP, Thematic Roles and Selectional Restrictions in Semantics, Word Senses and Relations Between Senses, **Processing Indian Languages in NLP.**

UNIT V: Speech Processing and Advanced NLP Models

lectures: 9 hours

Speech Fundamentals: Phonetics and Acoustic Phonetics, Digital Signal Processing in Speech Analysis, Feature Extraction in Speech: Short-Time Fourier Transform (STFT), Mel-Frequency Cepstral Coefficients (MFCC) and Perceptual Linear Prediction (PLP), Hidden Markov Models (HMMs) in Speech Recognition, **Introduction to prompt Engineering, LLM.**

Textbooks (Core Learning Materials)

1. Daniel Jurafsky & James H. Martin – Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, Pearson Education, 2023.
2. Tanveer Siddiqui & U.S. Tiwary – Natural Language Processing and Information Retrieval, Oxford University Press.

Reference Books (Supplementary Learning)

1. T.V. Geetha – Understanding Natural Language Processing – Machine Learning and Deep Learning Perspectives, Pearson, 2024.
2. Akshay Kulkarni & Adarsha Shivananda – Natural Language Processing Recipes - Unlocking Text Data with Machine Learning and Deep Learning using Python, Apress, 2019.

Parts-of-Speech (POS) Tagging in NLP

What is POS Tagging?

Parts-of-Speech (POS) Tagging is the process of identifying the grammatical role of every word in a sentence and assigning a label (tag) to it, such as:

- **Noun (NN)** – Name of a person, place, or thing
- **Verb (VB)** – Action word
- **Adjective (JJ)** – Describes a noun
- **Adverb (RB)** – Describes a verb, adjective, or another adverb
- **Pronoun (PRP)** – Replaces a noun
- **Preposition (IN)** – Shows relationship
- **Determiner (DT)** – Words like *the*, *a*, *an*

It helps computers understand the structure and meaning of human language.

Simple Definition

POS Tagging is the process of assigning a grammatical category to each word in a sentence.

Why is POS Tagging Important?

It helps computers understand:

- Who is doing the action?
- What is the action?
- Which words describe other words?

This improves many NLP applications such as:

- Machine Translation
- Chatbots
- Sentiment Analysis
- Search Engines
- Information Retrieval
- Text Classification

Example

Sentence:

The quick brown fox jumps over the lazy dog.

Word	POS Tag	Meaning
The	DT	Determiner
quick	JJ	Adjective
brown	JJ	Adjective
fox	NN	Noun
jumps	VBZ	Verb
over	IN	Preposition
the	DT	Determiner
lazy	JJ	Adjective
dog	NN	Noun

Explanation

- **fox** → noun because it is an animal.
- **quick** and **brown** → adjectives because they describe the fox.
- **jumps** → verb because it shows an action.
- **over** → preposition because it shows position.

- **the** → determiner because it specifies the noun.

1. Parts of Speech

These are grammatical categories like:

- Noun
- Verb
- Adjective
- Adverb
- Pronoun
- Preposition
- Determiner

2. Tagging

Assigning the correct grammatical label to every word.

3. Corpus

A **corpus** is a large collection of text used to train POS-tagging models.

Working of POS Tagging

Step 1: Tokenization

Split the sentence into words.

Example:

I love NLP



[I] [love] [NLP]

Step 2: Preprocessing

Clean the text by:

- converting to lowercase (if needed)
- removing unwanted symbols
- preparing the sentence for processing

Step 3: Load a Language Model

Libraries such as:

- NLTK
- SpaCy

use pre-trained language models that know English grammar.

Step 4: Linguistic Analysis

The model examines surrounding words to understand the role of each word.

Example:

Book a ticket.

Here **Book** is a **verb**.

This is a book.

Here **book** is a **noun**.

The context changes the tag.

Step 5: POS Tagging

Assign the correct tag to each word.

Example:

She eats mangoes.

Word	Tag
She	PRP
eats	VBZ
mangoes	NNS

Step 6: Evaluation

Check whether the assigned tags are correct and improve the model if necessary.

Types of POS Tagging

1. Rule-Based POS Tagging

Idea

Uses predefined grammar rules.

Example rule:

Words ending with **-tion** are usually nouns.

Sentence:

The presentation was excellent.

Word	Tag
presentation	Noun
was	Verb
excellent	Adjective

Advantages

- Easy to understand
- Good for simple grammar

Disadvantages

- Cannot handle many exceptions
- Needs many manually written rules

2. Transformation-Based Tagging (TBT)

Idea

Starts with initial tags and then corrects them using context-based rules.

Example:

Sentence:

The cat chased the mouse.

Initially:

The → DET

cat → NOUN

chased → VERB

If a transformation rule applies, the tag can be changed based on surrounding words.

3. Statistical POS Tagging

Idea

Uses probabilities learned from a large corpus to predict the most likely tag.

Common models:

- Hidden Markov Models (HMM)

- Conditional Random Fields (CRF)

Example

Sentence:

I can swim.

The model predicts **can** is a **modal verb**.

Sentence:

I bought a can.

Now **can** is predicted as a **noun** because of its context.

POS Tagging Using Python

The document shows two popular libraries:

1. NLTK

Steps:

1. Install NLTK.
2. Download required datasets.
3. Tokenize the sentence.
4. Apply `pos_tag()`.
5. Display the tagged words.

2. SpaCy

Steps:

1. Install SpaCy.
2. Load the English model.
3. Process the text.
4. Print each word with its POS tag.

Advantages

- Makes complex sentences easier to understand.
- Improves search and information retrieval.
- Helps Named Entity Recognition (NER).
- Supports machine translation and text analysis.

Challenges

1. Ambiguity

The same word can have different meanings.

Example:

- **Book** a ticket. → Verb
- Read a **book**. → Noun

2. Idiomatic Expressions

Expressions like:

"Break a leg."

do not mean their literal words, making tagging harder.

3. Out-of-Vocabulary Words

New or unknown words may be tagged incorrectly.

4. Domain Dependence

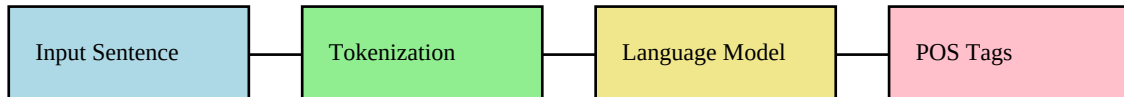
A model trained on news articles may not perform well on medical or legal text.

Concept	Explanation
POS Tagging	Assigns a grammatical tag to every word
Purpose	Helps computers understand sentence structure
Input	Sentence
Output	Words with grammatical tags
Common Tags	NN, VB, JJ, RB, DT, IN, PRP
Methods	Rule-Based, Transformation-Based, Statistical
Applications	Chatbots, Translation, Sentiment Analysis, Search Engines
Challenges	Ambiguity, idioms, unknown words, domain dependence

POS Tagging in NLP

Parts-of-Speech (POS) Tagging assigns grammatical labels such as noun, verb, adjective and adverb to each word in a sentence.

Flowchart



Example POS Tags

Word	Tag
The	DT
quick	JJ
fox	NN
jumps	VBZ
over	IN
dog	NN

Advantages

- Improves machine translation
- Supports chatbots
- Better search engines
- Helps sentiment analysis

Maximum Entropy Model (MaxEnt) for POS Tagging

A Maximum Entropy (MaxEnt) model is a machine learning model that predicts POS tags using many different features instead of relying only on probabilities between neighboring tags. Maximum Entropy is a feature-based **statistical model** that predicts the most suitable POS tag using contextual information.

Features Used The model considers features such as:

The model considers features such as:

- Current word
- Previous word
- Next word
- Prefixes
- Suffixes
- Capitalization
- Numbers
- Previous
- POS tag

Example Sentence:

She is reading a book.

For the word **reading**, the model uses:

- Previous word → is
- Next word → a
- Word ending → ing
- Previous tag → AUX

Based on these features,

it predicts: **reading → VERB**

Maximum Entropy principle

$$P(y|x) = \frac{1}{Z(x)} \exp \left(\sum_i w_i f_i(x, y) \right)$$

where:

- x = input/context
- y = possible POS tag
- $f_i(x, y)$ = feature function
- w_i = weight of the feature
- $Z(x)$ = normalization factor

The tag having the **highest probability** is selected.

1. $f_i(x, y)$ = Feature Function

A **feature function** is a rule that checks whether some useful information is present.

It usually gives:

$$f_i(x, y) = \begin{cases} 1 & \text{feature is present} \\ 0 & \text{feature is absent} \end{cases}$$

Example

For the word **playing**, we can define:

$$f_1(x, y) = 1$$

if:

The word ends in **"ing"** and the candidate POS tag is **VBG**.

For:

playing → VBG

the feature is true:

$$f_1(x, VBG) = 1$$

If the candidate tag is:

NN

then:

$$f_1(x, NN) = 0$$

Therefore, the feature provides evidence specifically for VBG.

2. w_i = Weight of the Feature

The **weight** tells us how important that feature is.

Suppose:

$$f_1(x, y) = 1$$

for:

ends with "ing" → VBG

The trained model might assign:

$$w_1 = 2.5$$

This means the feature gives strong positive evidence for VBG.

Another feature might have:

$$w_2 = 1.8$$

and another:

$$w_3 = -0.5$$

Weight	Meaning
+2.5	Strongly supports the tag
+0.5	Weakly supports the tag
0	No contribution
-0.5	Discourages the tag
-2.5	Strongly discourages the tag

The weights are **learned automatically from training data**.

3. $w_i f_i(x,y)$

This is important because the model doesn't use the weight alone.

It calculates:

$$w_i f_i(x, y)$$

Suppose:

$$w_1 = 2.5$$

and:

$$f_1(x, VBG) = 1$$

Then:

$$w_1 f_1 = (2.5)(1) = 2.5$$

So this feature contributes **+2.5** to the score of VBG.

If the feature is absent:

$$f_1(x, VBG) = 0$$

then:

$$w_1 f_1 = (2.5)(0) = 0$$

So the feature contributes nothing.

4. Multiple features

Usually, MaxEnt does not use only one feature.

Suppose for **playing** → VBG we have:

Feature	f_i	w_i
Ends with "ing"	1	2.5
Previous word is "is"	1	1.8
Next word is "cricket"	1	0.7
Starts with capital	0	-0.5

The model calculates:

$$\sum_i w_i f_i$$

Therefore:

$$\begin{aligned} & (2.5)(1) + (1.8)(1) + (0.7)(1) + (-0.5)(0) \\ &= 2.5 + 1.8 + 0.7 \\ &= \boxed{5.0} \end{aligned}$$

5. $Z(x)$ = Normalization Factor

Now we come to:

$$Z(x)$$

It is called the **normalization factor** or **partition function**.

Its purpose is to convert the scores of all possible POS tags into proper probabilities.

The model calculates:

$$Z(x) = \sum_{y'} \exp \left(\sum_i w_i f_i(x, y') \right)$$

Example

POS tag	Score
NN	1.0
VB	2.0
VBG	5.0
JJ	0.5

Now calculate exponential values:

$$e^1 = 2.72$$

$$e^2 = 7.39$$

$$e^5 = 148.41$$

$$e^{0.5} = 1.65$$

Therefore:

$$Z(x) = 2.72 + 7.39 + 148.41 + 1.65$$

$$Z(x) = 160.17$$

6. Now calculate the probability

For VBG:

$$P(VBG|x) = \frac{e^5}{Z(x)}$$

Therefore:

$$P(VBG|x) = \frac{148.41}{160.17}$$

$$\approx 0.927$$

or approximately:

$$92.7\%$$

Therefore, the model chooses:

$$\text{playing} \rightarrow \text{VBG}$$

Advantages

- **Uses many features simultaneously.**
- **More flexible than simple rule-based systems.**
- **Can model contextual information.**
- **Useful when linguistic rules are difficult to write manually.**
- **Handles overlapping and interacting features effectively.**

Limitations

- **Requires a sufficiently large labeled training dataset.**
- **Training can be computationally expensive.**
- **Feature engineering may be required.**
- **Performance depends on the quality of the selected features.**
- **Modern neural models such as BiLSTM and Transformers can outperform traditional MaxEnt models in many settings.**

Implementing POS Tagging with NLTK

Step 1: Import the Required Libraries

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
```

Step 2: Download the Required NLTK Resources

Run these commands only once.

```
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
```

These datasets are required for:

- **punkt_tab** → Splits text into words (tokenization).
- **averaged_perceptron_tagger_eng** → Assigns POS tags to words.

Step 3: Write the Program

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

# Download required resources
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

# Input sentence
text = "NLTK is a powerful library for natural language processing."

# Tokenize the sentence
words = word_tokenize(text)

# Apply POS tagging
pos_tags = pos_tag(words)

# Display results
print("Original Text:")
print(text)

print("\nPOS Tagging Result:")
for word, tag in pos_tags:
    print(f"{word}: {tag}")
```

Sample Output

Original Text:
NLTK is a powerful library for natural language processing.

POS Tagging Result:
NLTK: NNP
is: VBZ
a: DT

```
powerful: JJ
library: NN
for: IN
natural: JJ
language: NN
processing: NN
.: .
```

(The exact tags may vary slightly depending on the NLTK version.)

Code	Explanation
<code>import nltk</code>	Imports the NLTK library.
<code>word_tokenize(text)</code>	Splits the sentence into individual words (tokens).
<code>pos_tag(words)</code>	Assigns a Part-of-Speech tag to each token.
<code>for word, tag in pos_tags:</code>	Loops through each word and its POS tag.
<code>print(f"{word}: {tag}")</code>	Displays each word with its corresponding POS tag.

Import NLTK Library



Download Required Resources



Input Sentence



Tokenize Sentence



Apply POS Tagging



Store Tagged Words



Display Original Text



Print Each Word with its POS Tag

Program

```
import nltk
```

Explanation

- `import` is used to include a Python library.
- `nltk` stands for **Natural Language Toolkit**.
- It is a Python library used for **Natural Language Processing (NLP)** tasks such as tokenization, stemming, lemmatization, and POS tagging.

```
from nltk.tokenize import word_tokenize
```

Explanation

- `from` imports a specific function from a library.
- `nltk.tokenize` is the module used for breaking text into smaller units called **tokens**.
- `word_tokenize` is a function that splits a sentence into individual words and punctuation marks.

Example

```
text = "I love Python."
```

Output:

```
['I', 'love', 'Python', '.']
```

```
from nltk import pos_tag
```

Explanation

- Imports the `pos_tag()` function.
- `pos_tag()` assigns a **Part-of-Speech (POS)** tag to each word in a sentence.

Example

```
['I', 'love', 'Python']
```

becomes

```
[('I', 'PRP'), ('love', 'VBP'), ('Python', 'NNP')]
```

```
# Download required resources
```

Explanation

- This is a **comment**.
- Anything after `#` is ignored by Python.
- It is used to explain the purpose of the code.

```
nltk.download('punkt_tab')
```

Explanation

- Downloads the **punkt tokenizer data** required for word tokenization.
- It helps `word_tokenize()` identify word boundaries correctly.
- You only need to download it once.

```
nltk.download('averaged_perceptron_tagger_eng')
```

Explanation

- Downloads the **English POS Tagger model**.
- This model has been trained on a large English text corpus.
- It is used by `pos_tag()` to assign grammatical tags.

```
# Input sentence
```

Explanation

- Another comment indicating that the next line stores the input sentence.

```
text = "NLTK is a powerful library for natural language processing."
```

Explanation

- Creates a variable named `text`.
- Stores the sentence as a **string**.

Variable:

Variable	Value
text	"NLTK is a powerful library for natural language processing."

```
# Tokenize the sentence
```

Explanation

- Comment explaining the next operation.

```
words = word_tokenize(text)
```

Explanation

- Calls the `word_tokenize()` function.
- Splits the sentence into individual words and punctuation.
- Stores the result in the variable `words`.

Output:

```
['NLTK',  
'is',  
'a',  
'powerful',
```

```
'library',  
'for',  
'natural',  
'language',  
'processing',  
'.']
```

```
# Apply POS tagging
```

Explanation

- Comment explaining that POS tagging will now be performed.

```
pos_tags = pos_tag(words)
```

Explanation

- Calls the `pos_tag()` function.
- Assigns a grammatical tag to every token.
- Stores the result in `pos_tags`.

Output:

```
[  
( 'NLTK', 'NNP'),  
( 'is', 'VBZ'),  
( 'a', 'DT'),  
( 'powerful', 'JJ'),  
( 'library', 'NN'),  
( 'for', 'IN'),  
( 'natural', 'JJ'),  
( 'language', 'NN'),  
( 'processing', 'NN'),  
( '.', '.' )  
]
```

```
# Display results
```

Explanation

- Comment indicating that the program will display the output.

```
print("Original Text:")
```

Explanation

- Prints the heading **Original Text:** on the screen.

Output

Original Text:

```
print(text)
```

Explanation

- Prints the original sentence stored in the `text` variable.

Output

NLTK is a powerful library for natural language processing.

```
print("\nPOS Tagging Result:")
```

Explanation

- `\n` means **new line**.
- Moves the cursor to the next line before printing the heading.

Output

POS Tagging Result:

```
for word, tag in pos_tags:
```

Explanation

- `for` creates a loop.
- It goes through every `(word, tag)` pair in the `pos_tags` list.
- During each iteration:
 - `word` stores the token.
 - `tag` stores its POS tag.

Example:

First iteration

```
word = NLTK  
tag = NNP
```

Second iteration

```
word = is  
tag = VBZ
```

The loop continues until all tokens have been processed.

```
print(f"{word}: {tag}")
```

Explanation

- `f` indicates an **f-string** (formatted string).
- `{word}` is replaced with the current word.
- `{tag}` is replaced with its POS tag.

Example

If

```
word = powerful  
tag = JJ
```

Output

```
powerful: JJ
```

Explanation:

- `from` → Python keyword
- `nltk` → Module (library)
- `import` → Python keyword
- `pos_tag` → Function

Code	Type	How to Identify
<code>import</code>	Keyword	Reserved word in Python used to import modules.
<code>from</code>	Keyword	Reserved word used to import specific items from a module.
<code>import</code> (second occurrence)	Keyword	Used with <code>from</code> to import specific functions.
<code>nltk</code>	Module (Library)	Imported library that provides NLP tools.
<code>tokenize</code>	Module	A submodule inside the <code>nltk</code> package.
<code>word_tokenize</code>	Function	Performs an action (splits text into words). Usually followed by <code>()</code> .
<code>pos_tag</code>	Function	Assigns POS tags. Followed by <code>()</code> .
<code>download</code>	Function/Method	Called as <code>nltk.download()</code> . Performs downloading.
<code>text</code>	Variable	Stores the input sentence.
<code>words</code>	Variable	Stores the list of tokens.
<code>pos_tags</code>	Variable	Stores the tagged output.
<code>print</code>	Built-in Function	Displays output on the screen.

Installing spaCy

Step 1: Install spaCy

```
pip install spacy
```

Step 2: Download the English Language Model

```
python -m spacy download en_core_web_sm
```

Python Program for POS Tagging

```
import spacy
```

```
# Load English language model
```

```
nlp = spacy.load("en_core_web_sm")
```

```
# Input sentence
```

```
text = "NLTK is a powerful library for natural  
language processing."
```

```
# Process the text
```

```
doc = nlp(text)
```

```
# Print POS tags
```

```
print("Word\t\tPOS")
```

```
for token in doc:
```

```
print(f"{token.text}\t\t{token.pos_}")
```

Sample Output

Word	POS Tag	Meaning
NLTK	PROPN	Proper Noun
is	AUX	Auxiliary Verb
a	DET	Determiner
powerful	ADJ	Adjective
library	NOUN	Noun
for	ADP	Adposition (Preposition)
natural	ADJ	Adjective
language	NOUN	Noun
processing	NOUN	Noun
.	PUNCT	Punctuation

Another Example

```
import spacy
nlp = spacy.load("en_core_web_sm")
text = "The boy is playing football in the park."
doc = nlp(text)
for token in doc:
    print(token.text, ":", token.pos_)
```

Output

The : DET

boy : NOUN

is : AUX

playing : VERB

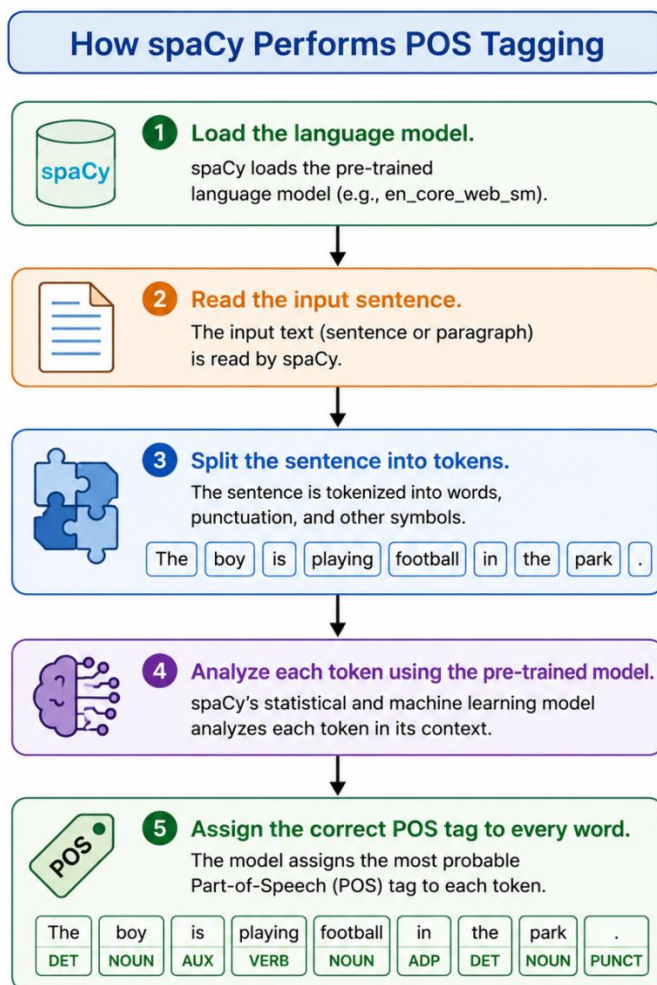
football : NOUN

in : ADP

the : DET

park : NOUN

.: PUNCT



Below is a **line-by-line explanation** of the spaCy POS Tagging program.

```
import spacy
```

Explanation:

- **import** → A Python keyword used to import a library or module.
 - **spacy** → The NLP (Natural Language Processing) library.
 - This statement imports the **spaCy library** so that its functions can be used in the program.
-

```
nlp = spacy.load("en_core_web_sm")
```

Explanation:

- **nlp** → A variable that stores the English language model.
- **=** → Assignment operator; assigns the loaded model to the variable nlp.
- **spacy.load()** → A spaCy function that loads a pre-trained language model.
- **"en_core_web_sm"** → The small English language model provided by spaCy.
 - **en** = English
 - **core** = Core language model
 - **web** = Trained on web text

- **sm** = Small model

Purpose: Loads the English NLP model so it can analyze text.

```
text = "The boy is playing football in the park."
```

Explanation:

- **text** → Variable that stores the input sentence.
- **=** → Assigns the sentence to the variable.
- **"The boy is playing football in the park."** → Input text that will be analyzed.

Purpose: Stores the sentence for processing.

```
doc = nlp(text)
```

Explanation:

- **doc** → Variable that stores the processed document.
- **nlp(text)** → Sends the input sentence to the spaCy model for analysis.

spaCy performs several NLP tasks automatically, such as:

- Tokenization
- POS Tagging
- Lemmatization
- Dependency Parsing

- Named Entity Recognition (NER)

Purpose: Converts the sentence into a spaCy **Doc object**, which contains all linguistic information.

for token in doc:

Explanation:

- **for** → Python loop keyword.
- **token** → Variable representing one word (token) at a time.
- **in** → Checks each item inside doc.
- **doc** → The processed document.

Purpose: Iterates through every word in the sentence one by one.

For this sentence, the tokens are:

The
boy
is
playing
football
in
the
park
.

```
print(token.text, ":", token.pos_)
```

Explanation:

- **print()** → Displays output on the screen.
- **token.text** → Prints the actual word.
- **":"** → Prints a colon for better formatting.
- **token.pos_** → Prints the Part-of-Speech (POS) tag assigned by spaCy.

Purpose: Displays each word with its grammatical category.

Hidden Markov Models (HMM) and Maximum Entropy Models for POS Tagging

1. Hidden Markov Model (HMM) for POS Tagging

Definition

A **Hidden Markov Model (HMM)** is a **statistical model** used for POS tagging. It predicts the **most likely sequence of Part-of-Speech (POS) tags** for a sentence based on probabilities.

HMM is a probabilistic model that assigns the most likely POS tag sequence to a sentence using transition and emission probabilities.

Why is it called "Hidden"?

- The **words** in the sentence are **visible (observed)**.
- The **POS tags** are **hidden** because they are not given directly and must be predicted.

Example

Sentence:

The boy is playing football.

Observed Words:

The → boy → is → playing → football

Hidden POS Tags:

DET → NOUN → AUX → VERB → NOUN

Components of HMM

1. States

The hidden states are the **POS tags**.

Examples:

- Noun
- Verb
- Adjective
- Adverb
- Pronoun

2. Observations

The observations are the **words** in the sentence.

Example:

dog

cat

runs

beautiful

3. Transition Probability

Probability of moving from one POS tag to another.

Example:

$P(\text{Verb} \mid \text{Noun})$

Meaning:

Probability that a **Verb** comes after a **Noun**.

4. Emission Probability

Probability of a word being generated from a POS tag.

Example:

$P(\text{dog} \mid \text{Noun})$

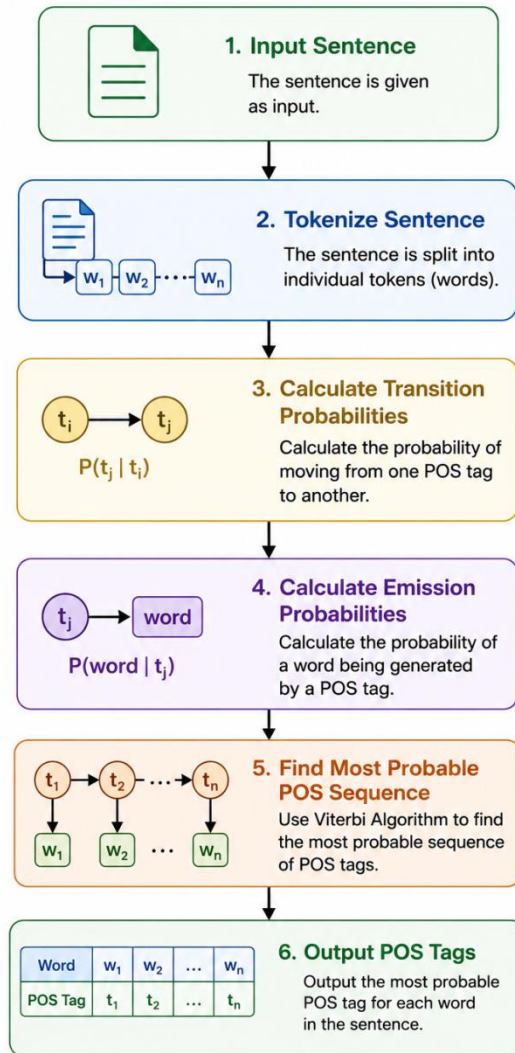
Meaning:

Probability that the word **dog** is a **Noun**.

Working of HMM

1. Read the sentence.
2. Consider all possible POS tag sequences.
3. Calculate transition probabilities.
4. Calculate emission probabilities.
5. Select the sequence with the highest probability.

HMM-Based POS Tagging Process



Advantages of HMM

- High accuracy.
- Uses context.
- Handles ambiguity.
- Widely used in NLP.

Disadvantages of HMM

- Requires large training data.
- Assumes dependence only on the previous tag.
- Cannot easily capture long-distance relationships.

2. Maximum Entropy Model (MaxEnt) for POS Tagging

Definition

A **Maximum Entropy (MaxEnt)** model is a **machine learning model** that predicts POS tags using many different features instead of relying only on probabilities between neighboring tags.

Maximum Entropy is a feature-based statistical model that predicts the most suitable POS tag using contextual information.

Features Used

The model considers features such as:

- Current word
- Previous word
- Next word
- Prefixes

- Suffixes
- Capitalization
- Numbers
- Previous POS tag

Example

Sentence:

She is reading a book.

For the word **reading**, the model uses:

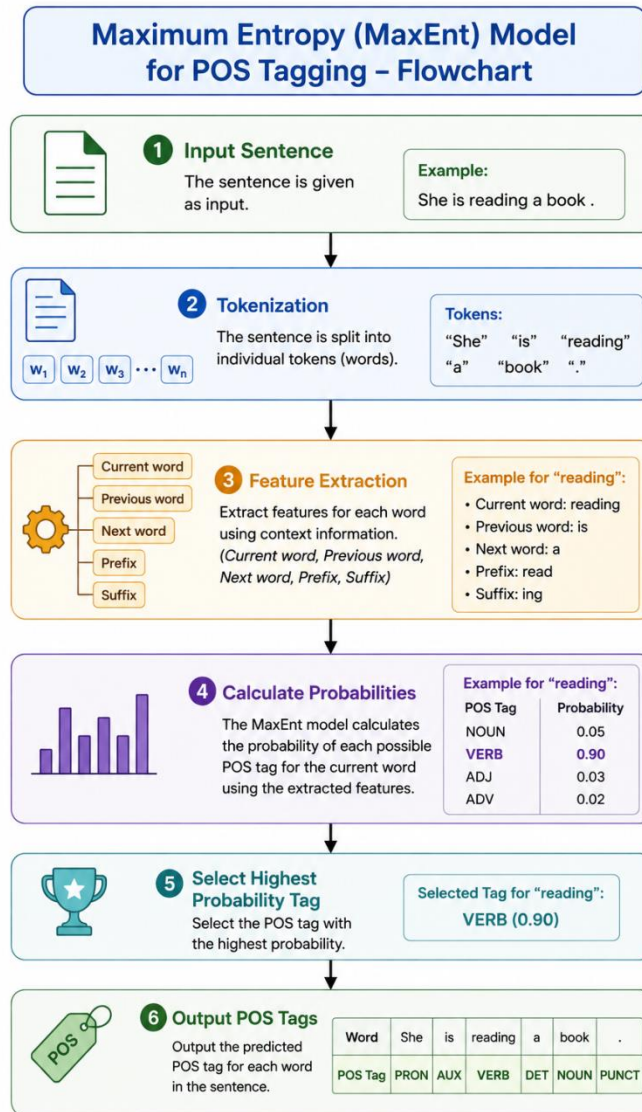
- Previous word → is
- Next word → a
- Word ending → ing
- Previous tag → AUX

Based on these features, it predicts:

reading → VERB

Working of Maximum Entropy Model

1. Read the sentence.
2. Extract features.
3. Calculate probabilities for each POS tag.
4. Choose the tag with the highest probability.



Advantages

- Uses many contextual features.
- More accurate than simple probabilistic models.
- Handles unknown words better.
- Flexible and robust.

Disadvantages

- Slower than HMM.
- Requires feature engineering.
- Needs more computation.

Feature	Hidden Markov Model (HMM)	Maximum Entropy (MaxEnt)
Model Type	Statistical (Probabilistic)	Machine Learning (Feature-based)
Decision Basis	Transition & emission probabilities	Multiple contextual features
Uses Context	Limited	Extensive
Handles Unknown Words	Moderate	Better
Speed	Faster	Slower
Accuracy	Good	Higher (with good features)

Real-Life Applications

- Grammar checking
- Machine translation
- Speech recognition
- Chatbots
- Search engines
- Information extraction

Context-Free Grammar (CFG)

A **Context-Free Grammar (CFG)** is a formal grammar used to describe the syntactic structure of sentences in natural language.

A context-free grammar (CFG) is a formal system used to describe a class of languages known as context-free languages (CFLs). Purpose of context-free grammar is:

- To list all strings in a language using a set of rules (production rules).
- It extends the capabilities of regular expressions and finite automata.

A CFG (or just a grammar) G is a tuple

$$G = (V, \Sigma, R, S)$$

where:

- $V \rightarrow$ Set of non-terminal symbols
- $\Sigma \rightarrow$ Set of terminal symbols
- $R \rightarrow$ Set of production rules
- $S \rightarrow$ Start symbol

Think of these four components as the building blocks of a grammar.

Example sentence

The cat eats fish.

We can define a simple CFG for this sentence.

1. $V \rightarrow$ Set of Non-Terminal Symbols

Non-terminals are symbols that represent grammatical categories or phrases. They can be further expanded using grammar rules.

For example:

$V = \{S, NP, VP, Det, N, V\}$

Where:

- S = Sentence
- NP = Noun Phrase
- VP = Verb Phrase
- Det = Determiner
- N = Noun
- V = Verb

For example:

$S \rightarrow NP VP$

Here, S is expanded into NP + VP.

Non-terminal = category that still needs to be expanded.

2. $\Sigma \rightarrow$ Set of Terminal Symbols

Terminals are the actual words or symbols that appear in the final sentence.

For our example:

The cat eats fish.

We can have:

$\Sigma = \{\text{the, cat, eats, fish}\}$

These symbols cannot be expanded further by grammar rules.

For example:

$\text{Det} \rightarrow \text{the} \quad \text{N} \rightarrow \text{cat} \quad \text{V} \rightarrow \text{eats}$

Terminal = final word appearing in the sentence.

3. $R \rightarrow$ Set of Production Rules

Production rules specify how non-terminals can be replaced or expanded.

For example:

$$R = \begin{cases} S \rightarrow NP VP \\ NP \rightarrow Det N \\ VP \rightarrow V NP \\ Det \rightarrow the \\ N \rightarrow cat \mid fish \\ V \rightarrow eats \end{cases}$$

These rules tell us how to construct a valid sentence.

For example:

$$S \rightarrow NP VP$$

Then:

$$NP \rightarrow Det N$$

and:

$$Det \rightarrow the \quad N \rightarrow cat$$

So:

$$NP \Rightarrow Det \ N \Rightarrow the \ cat$$

4. $S \rightarrow$ Start Symbol

S is the starting point of the grammar.

Usually:

S=Sentence

We start with:

S

Then apply the rules:

$$S \rightarrow NP VP \quad NP \rightarrow Det N \quad VP \rightarrow V NP$$

Eventually:

$$S \Rightarrow NP VP \Rightarrow Det N V NP \Rightarrow the \ cat \ eats \ fish$$

Therefore:

The cat eats fish.

S = Starting point from which the sentence is generated.

Symbol	Meaning	Simple explanation
V	Non-terminals	Grammar categories that can be expanded
Σ	Terminals	Actual words in the sentence
R	Production Rules	Rules used to generate the sentence
S	Start Symbol	Starting point, usually Sentence

CFG = Categories + Words + Rules + Start
--

What is Parsing?

Parsing is the process of analyzing a sentence to understand its structure and meaning.

In Natural Language Processing (NLP), parsing helps a computer to break a sentence into parts and check how these parts are related.

When we say: “John plays football.”

Parsing helps the computer to understand:

- John → Who is doing the action (Subject)
- plays → What is the action (Verb)
- football → On what (Object)

Why Do We Need Parsing?

- To check if a sentence is grammatically correct
- To find the meaning and structure of the sentence
- To help machines understand human language

Constituency Parsing:

- [NP The boy] [VP eats [NP an apple]]
- Shows who, what action, and what object

Dependency Parsing:

- “eats” is the main verb
- “boy” → subject of “eats”
- “apple” → object of “eats”
- “an” → describes “apple”

Where Parsing is Used in Real Life?

Google Search – To understand what you are searching for
Chatbots – To reply correctly based on your sentence
Grammar Checkers – To detect wrong sentences
Voice Assistants – To perform correct actions based on your command
Machine Translation (Google Translate) – To translate sentences in correct grammar

Constituency Parsing (CP)

Constituency Parsing helps to break a sentence into smaller parts like **noun phrases (NP)**, **verb phrases (VP)**, and other groups. These parts are called **constituents**. It also checks **how these parts are connected**, like a family tree. The result is a **parse tree** showing the structure of the sentence.

Real-Time Example:

- Sentence: "**The little boy plays in the park.**"
- Constituency Parse:
 - [S
 - [NP The little boy]
 - [VP plays

- [PP in the park]
 -]
 -]
- It shows the sentence (S) has a **Noun Phrase (NP)** and a **Verb Phrase (VP)**.

How Constituency Parsing Works:

- It uses grammar rules like:
 - A **sentence (S)** = **Noun Phrase (NP)** + **Verb Phrase (VP)**
 - A **Noun Phrase (NP)** = **Article** + **Adjective** + **Noun**
- The parser checks the words in the sentence and makes a tree using these rules.

Applications of Constituency Parsing:

1. Natural Language Processing (NLP)

- Used in tasks like **text summarization**, **machine translation**, and **question answering**.
- Example: In summarizing a news article, CP helps pick out the main subjects and actions.

2. Information Retrieval

- Used in search engines to pull correct information.
- Example: Searching "best places to visit in Switzerland" retrieves correct results based on sentence structure.

3. Text-to-Speech

- Helps computers speak naturally by understanding the sentence flow.
- Example: Siri uses this to read your messages naturally.

4. Sentiment Analysis

- Finds positive/negative words inside sentence parts.
- Example: "The movie was not good." — The parser catches that "not good" is a negative phrase.

5. Chatbots and Text-based Games

- Helps bots reply better.
- Example: In a game, if you type "Pick up the sword," the bot understands "Pick up" is an action.

6. **Text Summarization**

- Picks key parts of text for short summaries.
- Example: Summarizing a long article into a short paragraph.

7. **Text Classification**

- Helps classify text into topics (sports, politics).
- Example: Sorting emails into "Work" or "Personal."

Constituency Parsing (CP) – Line-by-Line Program Explanation

Python Program

```
import nltk
from nltk import CFG

grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'an'
N -> 'boy' | 'apple'
V -> 'eats'
""")

parser = nltk.ChartParser(grammar)

sentence = "the boy eats an apple".split()

for tree in parser.parse(sentence):
    print(tree)
    tree.pretty_print()
```

1. import nltk

`nltk` means Natural Language Toolkit. It is a Python library used for Natural Language Processing (NLP).

2. from nltk import CFG

`CFG` stands for Context-Free Grammar. It is used to define grammar rules such as $S \rightarrow NP VP$.

3. grammar = CFG.fromstring(...)

`CFG.fromstring()` creates a Context-Free Grammar from the rules written as text. The variable `grammar` stores the grammar.

4. S -> NP VP

`S` means Sentence, `NP` means Noun Phrase, and `VP` means Verb Phrase. Therefore, Sentence $\rightarrow$ Noun Phrase + Verb Phrase.

5. NP -> Det N

This means Noun Phrase $\rightarrow$ Determiner + Noun. For example, "the boy" consists of Det = the and N = boy.

6. VP -> V NP

This means Verb Phrase $\rightarrow$ Verb + Noun Phrase. For example, "eats an apple" consists of V = eats and NP = an apple.

7. Det -> 'the' | 'an'

`Det` means Determiner. The symbol `|` means OR, so the determiner can be "the" or "an".

8. N -> 'boy' | 'apple'

`N` means Noun. The grammar allows "boy" or "apple" as nouns.

9. V -> 'eats'

`V` means Verb. The grammar allows the verb "eats".

10. parser = nltk.ChartParser(grammar)

`ChartParser` creates a parser using the grammar. It searches for a valid syntactic structure for the input sentence.

11. sentence = "the boy eats an apple".split()

The sentence is divided into individual words. The result is ['the', 'boy', 'eats', 'an', 'apple'].

12. for tree in parser.parse(sentence):

The parser tries to generate valid parse trees for the sentence. The loop processes each tree found.

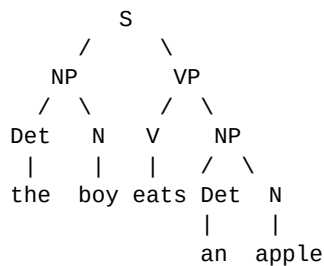
13. print(tree)

This prints the constituency structure in bracket notation, for example: (S (NP (Det the) (N boy)) (VP (V eats) (NP (Det an) (N apple)))).

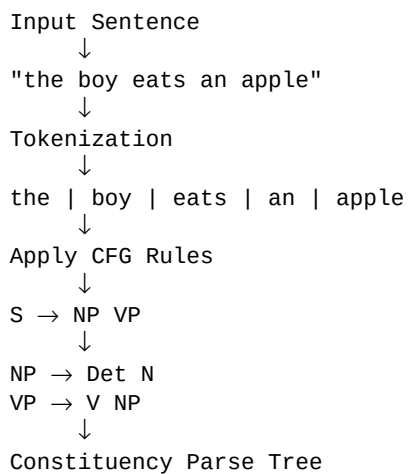
14. tree.pretty_print()

This displays the parse tree in a tree-like format, making the NP and VP structure easier to understand.

Expected Parse Tree



Complete Flow



Key idea: Constituency Parsing divides a sentence into meaningful phrases such as NP and VP and represents their structure using a parse tree.

Tree banks and NORMAL FORM GRAMMAR:

A **Treebank** is a collection of sentences that have been **manually or automatically parsed and annotated with syntactic or grammatical structure** in the form of trees.

We need TREEBANKS for computers to understand:

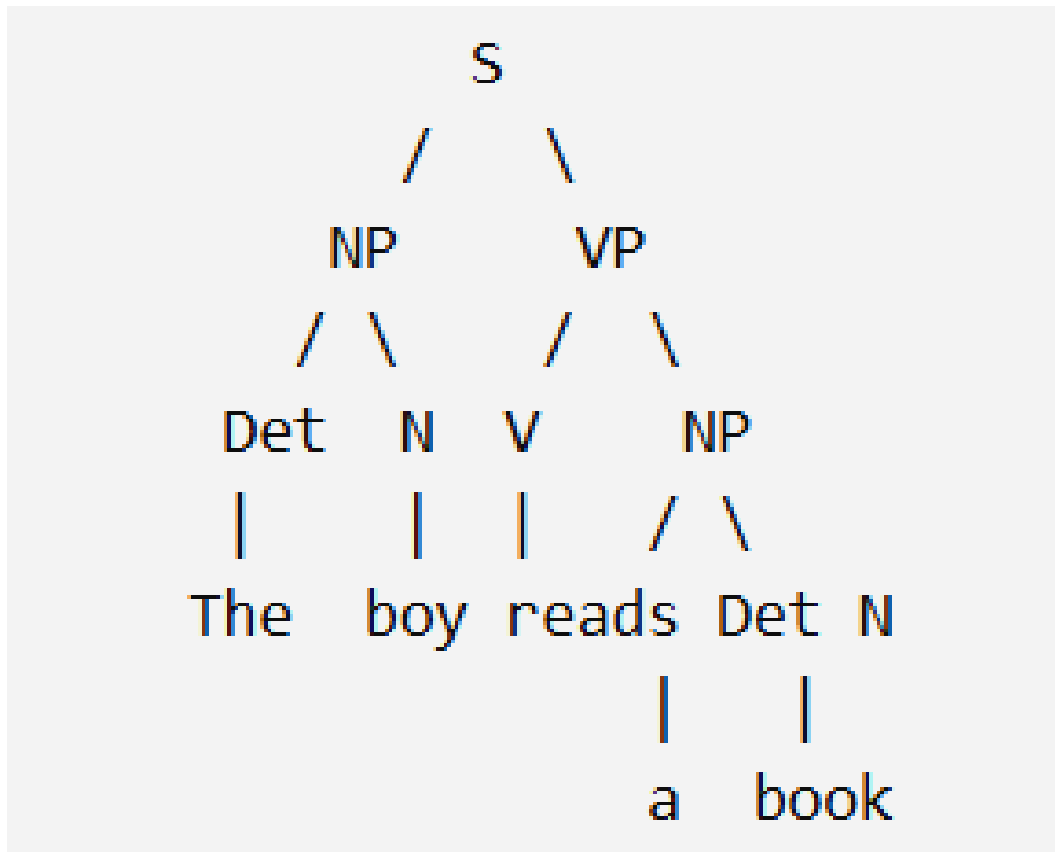
- Understand sentence structure
- Learn grammar from real examples
- Train **parsers** in NLP (parsers break down sentences)

WHAT IS THE REQUIREMENT OF TREE BANKS:

- Machines can't understand grammar naturally — they need examples.
- Treebanks provide **real sentence structures** so that NLP models can learn from them.
- Useful for training **parsers**, improving **grammar checking, machine translation, text summarization**, etc.

The boy reads a book.

The treebank stores the sentence together with information about its grammatical structure.



- **S = Sentence**
- **NP = Noun Phrase**
- **VP = Verb Phrase**
- **PP = Prepositional Phrase**
- **DT = Determiner**
- **N = Noun**
- **V = Verb**
- **P = Preposition**

Types of Treebanks

a. Constituency Treebank

A **Constituency Treebank** represents the **internal phrase structure of a sentence**. It shows how individual words are grouped together to form larger grammatical units called **constituents**.

Key Features

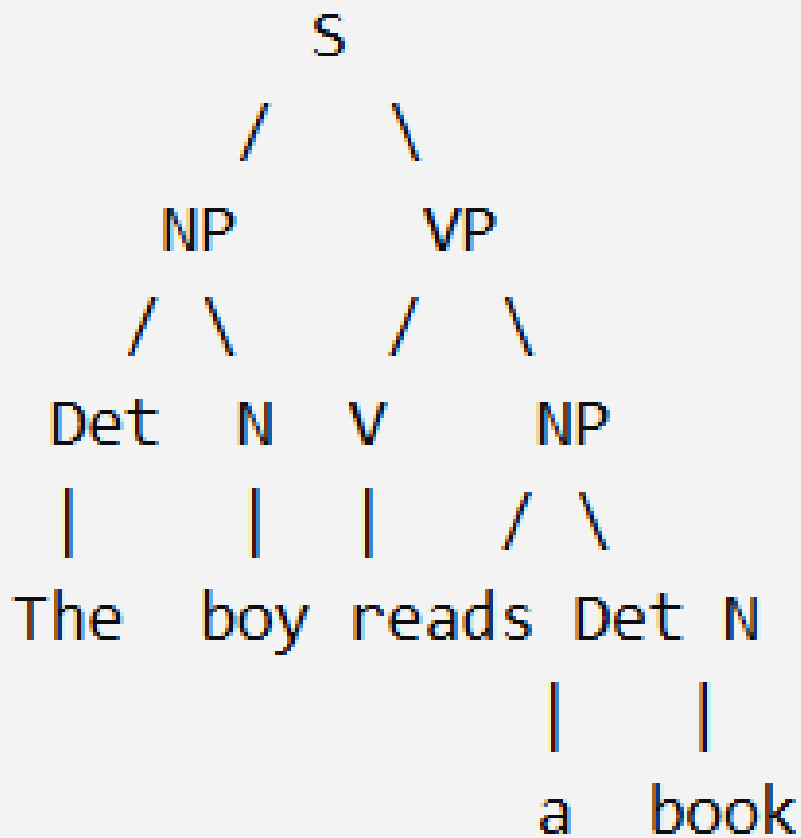
- Focuses on **how words are grouped into phrases**.
- Represents **nested phrase structures**.
- Uses grammatical categories such as:
 - **S** → Sentence
 - **NP** → Noun Phrase
 - **VP** → Verb Phrase
 - **PP** → Prepositional Phrase
 - **Det** → Determiner
 - **N** → Noun
 - **V** → Verb

Example

Sentence:

The boy reads a book.

Constituency structure:



Here, “**The boy**” forms an NP and “**a book**” forms another NP. The VP contains the verb “**reads**” and its object NP.

Example of a Constituency Treebank

Penn Treebank is a well-known example of a constituency-based treebank for English.

Simple way to remember:

Constituency Treebank = How words form phrases.

b. Dependency Treebank

A **Dependency Treebank** represents the **direct relationships between words** in a sentence.

Instead of grouping words into nested phrases, it focuses on which word is the **head** and which word depends on it.

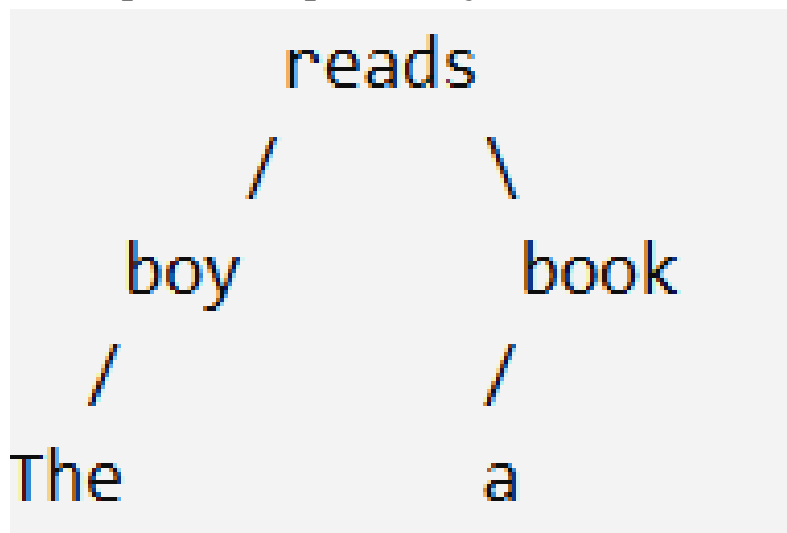
Key Features

- Shows how words are **directly connected**.
- Focuses on **head–dependent relationships**.
- Each dependent word is connected to its syntactic head.
- Represents the sentence primarily through **word-to-word relationships**.

Consider:

The boy reads a book.

A simplified dependency structure is:



Here:

- **reads** → main/head verb

- **boy** → subject dependent of *reads*
- **book** → object dependent of *reads*
- **The** → determiner dependent of *boy*
- **a** → determiner dependent of *book*

The important question is:

“Which word is related to which other word?”

Example of a Dependency Treebank

Universal Dependencies (UD) is a major multilingual framework for dependency-based syntactic annotation.

Simple way to remember:

Dependency Treebank = How words are related to each other.

Constituency vs Dependency Treebank

Feature	Constituency Treebank	Dependency Treebank
Main focus	Phrase structure	Word relationships
Basic unit	Constituent/Phrase	Word
Structure	Nested hierarchy	Head–dependent relations
Shows	How words form phrases	How words depend on other words
Example	Penn Treebank	Universal Dependencies
Typical representation	$S \rightarrow NP + VP$	Head → Dependent

```
import nltk  
from nltk import CFG  
from nltk.parse import ChartParser
```

```
# Define a Context-Free Grammar
```

```
grammar = CFG.fromstring("""  
    S -> NP VP  
    NP -> Det N  
    VP -> V NP  
    Det -> 'the' | 'a'  
    N -> 'boy' | 'book'  
    V -> 'reads'  
""")
```

```
# Create parser
```

```
parser = ChartParser(grammar)
```

```
# Input sentence
```

```
sentence = "the boy reads a book".split()
```

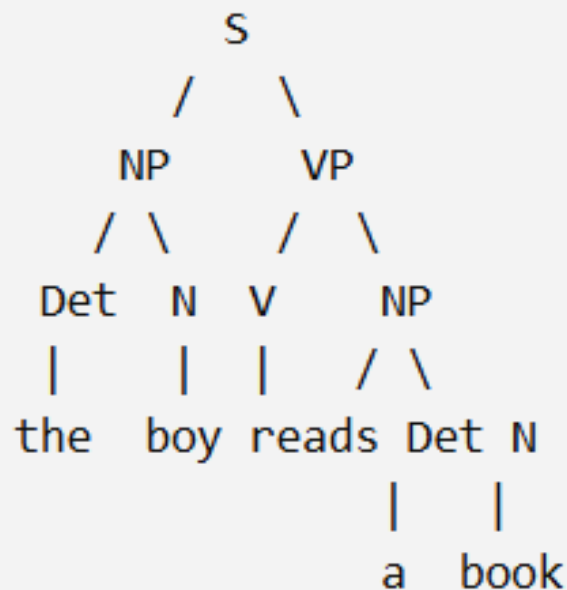
```
# Generate parse tree
```

```
for tree in parser.parse(sentence):
```

```
    print(tree)
```

```
    tree.pretty_print()
```

Output



Explanation

1. Import NLTK

```
import nltk
```

NLTK is a Python library used for Natural Language Processing.

2. Define the grammar

```
grammar = CFG.fromstring("""
    S -> NP VP
    NP -> Det N
    VP -> V NP
    """)
```

This defines the grammatical structure:

$S \rightarrow NP VP$

$NP \rightarrow Det\ N$

$VP \rightarrow V\ NP$

3. Create the parser

```
parser = ChartParser(grammar)
```

The ChartParser searches for valid parse trees using the given CFG.

4. Give the sentence

```
sentence = "the boy reads a book".split()
```

The sentence is divided into individual words.

5. Generate the tree

```
for tree in parser.parse(sentence):
```

The parser generates the possible constituency parse tree.

6. Display the tree

```
tree.pretty_print()
```

This displays the syntactic structure in a tree format.

CFG.fromstring(...)

“Read these grammar rules written as text and convert them into an NLTK CFG object.”

2. Very Simple Dependency Treebank Program

For a simple dependency representation, we can use **spaCy**:

```
import spacy  
# Load English NLP model  
nlp = spacy.load("en_core_web_sm")  
# Input sentence  
sentence = "The boy reads a book."  
# Process sentence  
doc = nlp(sentence)  
# Display dependency relationships  
for token in doc:  
    print(token.text, "->", token.dep_, "->", token.head.text)
```

Explanation

1. **import spacy**

Imports the **spaCy** library.

spaCy is an NLP library used for tasks such as:

- Tokenization
- POS tagging
- Named Entity Recognition
- Dependency Parsing
- Text classification

2. **nlp = spacy.load("en_core_web_sm")**

Loads the pre-trained English language model **en_core_web_sm**.

This model helps spaCy understand the grammatical structure of English sentences.

If the model is not installed, run:

```
python -m spacy download en_core_web_sm
```

```
3. sentence = "The boy reads a book."
```

Stores the input sentence in a variable called sentence.

The sentence contains five words:

```
The | boy | reads | a | book
```

```
4. doc = nlp(sentence)
```

Passes the sentence through the **spaCy** NLP pipeline.

spaCy analyzes the sentence and creates a Doc object containing linguistic information about every word.

```
5. for token in doc:
```

Loops through every word (token) in the sentence.

For example:

The

boy

reads

a

book

```
6. token.text
```

Displays the actual word.

For example:

Boy

```
7. token.dep_
```

Displays the **dependency relation** of the word.

Some common dependency labels are:

Dependency	Meaning
nsubj	Nominal subject
dobj	Direct object
det	Determiner
ROOT	Main/root word of sentence
punct	Punctuation

8. token.head.text

Displays the **head word** to which the current word is grammatically connected.

For example:

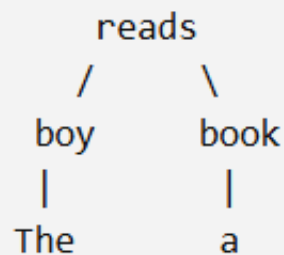
boy -> reads

means **boy** is grammatically dependent on **reads**.

Example output

The -> det -> boy
boy -> nsubj -> reads
reads -> ROOT -> reads
a -> det -> book
book -> dobj -> reads

Interpretation



Here:

- reads → ROOT
- boy → subject of reads
- book → object of reads
- The → determiner of boy
- a → determiner of book

NORMAL FORMS:

Normal Forms are **standard ways** of writing grammar rules.

A **normal form** is just a **standardized way of writing grammar rules**, so that:

- Computers can easily **parse** them
- They follow **specific patterns**
- They help design efficient parsers

Computers need **grammar rules** to be written in a **specific format** so they can process them easily and apply algorithms like **parsing**.

Normal Form = Packaging style

- Same **content** (grammar), but **different packaging** for **different purposes**

What is CNF (Chomsky Normal Form)?

A grammar is in CNF if:

Example 1:

- Every production is of the form:
 $A \rightarrow BC$ (two non-terminals)
or
 $A \rightarrow a$ (a single terminal)

Example 2:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

- **S** can be split into **A B**
- **A** gives **a**
- **B** gives **b**

So, **S** can finally become **a b**

RULES NOT ALLOWED:

- $A \rightarrow BCD$
- $A \rightarrow aB$
- $A \rightarrow \varepsilon$ (empty string)

Why to use CNF:

- Needed for **parsing algorithms** like **CYK Parser**
- Makes parsing **faster** and **consistent**
- Easy for computer programs to understand.

Treebanks	Normal Forms
Provide real examples of sentence trees	Give rules for building those trees
Help train parsers to read structure	Help define how parsing should work
Show sentence structure as trees	Show allowed formats for grammar rules

UNDERSTANDING THE PROCESS OF TREEBANK WITH (NORMAL FORMS).

Concept	What It Does	Example/Use
Treebank	Shows real sentence structures	Penn Treebank, UD
Constituency	Breaks sentence into phrase parts	$NP \rightarrow DT + NN$
Dependency	Shows word-to-word relationships	reads $\rightarrow$ subject: student, object: book
Normal Form	Standard rule format	$CNF \rightarrow A \rightarrow BC$ or $A \rightarrow a$
CNF	Needed for parsing algorithms	Used in CYK parser, simplified grammar handling

OTHER TYPE OF NORMAL FORMS:

Greibach Normal Form (GNF)

A grammar is in **Greibach Normal Form** if **every rule starts with a terminal symbol** (a word like ‘go’, ‘read’, ‘the’) followed by **zero or more non-terminals** (like Noun, Verb, etc.).

$$A \rightarrow aA_1A_2\dots A_n$$

- A is a non-terminal
- a is a **terminal** (actual word)
- $A_1, A_2, \dots, A_n$ are **non-terminals**.

We can't have rules like $A \rightarrow BC$ or $A \rightarrow \varepsilon$ (except in special cases)

$S \rightarrow aB$

$B \rightarrow b$

This is **already in GNF**, because:

- $S \rightarrow aB \rightarrow$ starts with terminal a
- $B \rightarrow b \rightarrow$ terminal b only

Kuroda Normal Form:

Kuroda Normal Form is used in **Type-0 grammars** (which are the most general and powerful grammars). It includes **context-sensitive** and **unrestricted grammars**.

Rules for the KNF:

$AB \rightarrow CD$ (2 non-terminals $\rightarrow$ 2 non-terminals)

$A \rightarrow BC$ (non-terminal $\rightarrow$ 2 non-terminals)

$A \rightarrow B$ (non-terminal $\rightarrow$ non-terminal)

$A \rightarrow a$ (non-terminal $\rightarrow$ terminal)

- It helps define grammars that can **recognize any computable language**.
- Mostly used in **theoretical computer science**, less in real NLP tools.

$AB \rightarrow CD$

If you find A followed by B , you can replace them with C followed by D

Normal Form	Rule Format	Usage	Starts With
CNF	$A \rightarrow BC$ or $A \rightarrow a$	Bottom-up parsers like CYK	Non-terminal or terminal
GNF	$A \rightarrow aA_1A_2...A_n$	Top-down parsers	Terminal
Kuroda NF	$AB \rightarrow CD$, $A \rightarrow BC$, etc.	Theoretical computing, Type-0	Mixed

```
from nltk import CFG
from nltk.treetransforms import chomsky_normal_form
from nltk import ChartParser
# Define a Context-Free Grammar
grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the'
N -> 'boy' | 'book'
V -> 'reads'
""")
print("Original Grammar:")
print(grammar)
# Create a parser
parser = ChartParser(grammar)
# Parse the sentence
sentence = "the boy reads the book".split()
# Display parse tree
print("\nParse Tree:")
for tree in parser.parse(sentence):
    print(tree)
# Convert a parse tree into Chomsky Normal Form
tree = list(parser.parse(sentence))[0]
chomsky_normal_form(tree)
print("\nChomsky Normal Form:")
print(tree)
```

Explanation

1. Import NLTK

from nltk import CFG

CFG is used to create a **Context-Free Grammar**.

2. Define the grammar

```
grammar = CFG.fromstring("""
```

```
S -> NP VP
```

```
NP -> Det N
```

```
VP -> V NP
```

```
Det -> 'the'
```

```
N -> 'boy' | 'book'
```

```
V -> 'reads'
```

```
""")
```

The grammar contains rules such as:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

Here:

- S = Sentence
 - NP = Noun Phrase
 - VP = Verb Phrase
 - Det = Determiner
 - N = Noun
 - V = Verb
-

3. Create the parser

```
parser = ChartParser(grammar)
```

The parser uses the grammar to analyze the sentence.

4. Input sentence

```
sentence = "the boy reads the book".split()
```

.split() separates the sentence into individual words:

the

boy

reads

the

book

5. Generate the parse tree

```
for tree in parser.parse(sentence):
```

```
    print(tree)
```

This produces a syntactic parse tree showing how the sentence is structured.

6. Convert to Chomsky Normal Form

```
chomsky_normal_form(tree)
```

This converts the **parse tree** into Chomsky Normal Form.

In CNF, grammar productions generally have the form:

$$A \rightarrow B C$$

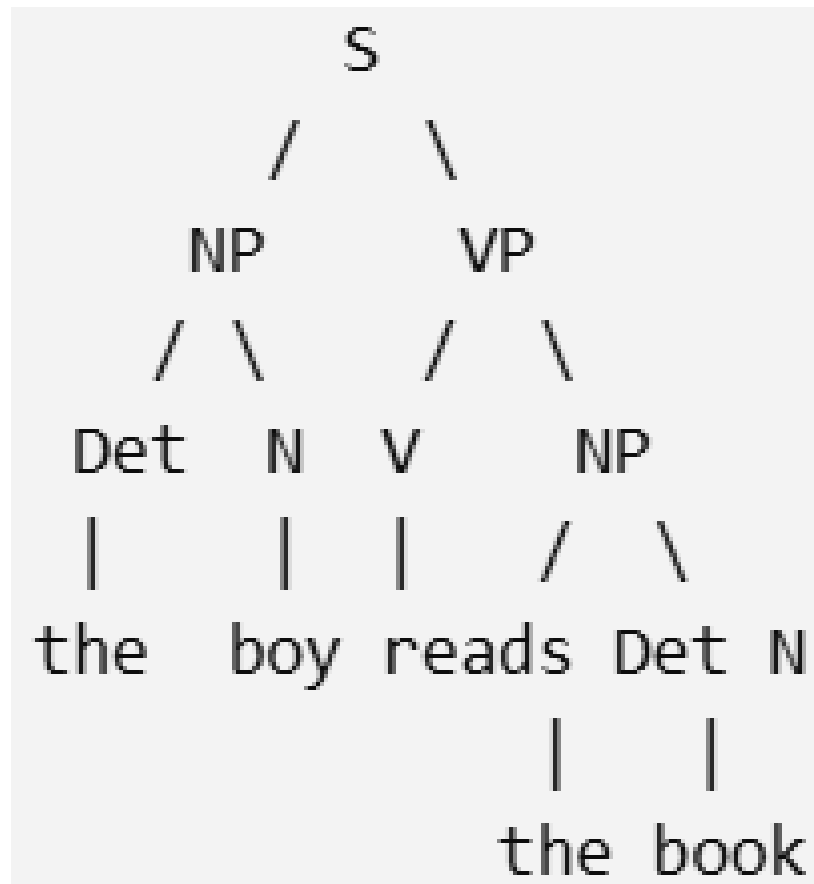
or

$$A \rightarrow \text{word}$$

where A, B, and C are grammar symbols.

Example

Original structure:



The important idea is:

CFG → Parse Tree → Chomsky Normal Form

Installation

If NLTK is not installed:

pip install nltk

Note: The `chomsky_normal_form()` function transforms a **parse tree**, not the original CFG object itself.

Top-Down and Bottom-Up Parsing Strategies

UNIT II: Word-Level and Syntactic Analysis – Natural Language Processing

1. Top-Down Parsing

Definition: Top-down parsing starts from the **start symbol of the grammar** and tries to generate the input sentence.

Basic idea

$S \rightarrow \text{Sentence}$

The parser starts with the root **S** and repeatedly expands non-terminals until it obtains the words in the input.

Example Grammar

$S \rightarrow NP VP$ $NP \rightarrow Det N$ $VP \rightarrow V NP$ $Det \rightarrow \text{the}$ $N \rightarrow \text{boy}$ $V \rightarrow \text{sees}$

For the sentence: "**the boy sees the boy**"

Step 1	S
Step 2	↓ NP VP
Step 3	↓ Det N VP
Step 4	↓ the boy VP
Step 5	↓ the boy V NP
Step 6	↓ the boy sees Det N
Step 7	↓ the boy sees the boy

Advantages

- Starts from the start symbol.
- Easy to understand and implement.
- Useful for grammars where the expected structure is known.
- Naturally constructs a parse tree from the root.

Disadvantages

- Can explore rules that do not match the input.
- May suffer from left recursion.

- Can involve considerable backtracking.
- May be inefficient for ambiguous grammars.

2. Bottom-Up Parsing

Definition: Bottom-up parsing works in the opposite direction. It starts with the **input words** and combines them into phrases until it reaches the start symbol **S**.

Basic idea

Words → Phrases → S

For the sentence: **"the boy sees the boy"**

Step 1	the boy sees the boy
Step 2	↓ Det N V Det N
Step 3	↓ NP V NP
Step 4	↓ NP VP
Step 5	↓ S

General process

Words → Det, N, V → NP, VP → S

Advantages

- Starts directly from the input.
- Does not predict structures that cannot be formed from the input.
- Handles left-recursive grammars better.
- Useful for constructing parse trees from observed words.

Disadvantages

- May generate many intermediate structures.
- Can be computationally expensive.
- Ambiguous sentences may produce several possible parse trees.

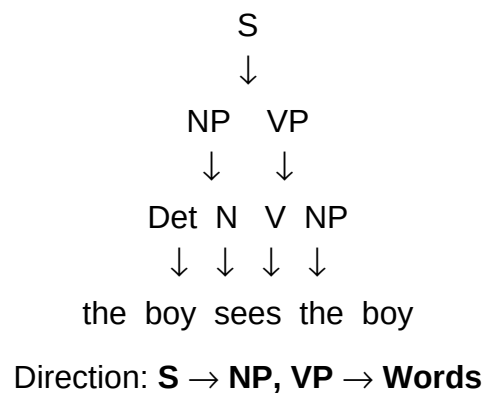
3. Top-Down vs Bottom-Up Parsing

Feature	Top-Down	Bottom-Up
Starting point	Start symbol S	Input words
Direction	Root → Leaves	Leaves → Root
Main operation	Expand	Reduce / Combine
Goal	Generate the sentence	Build the start symbol

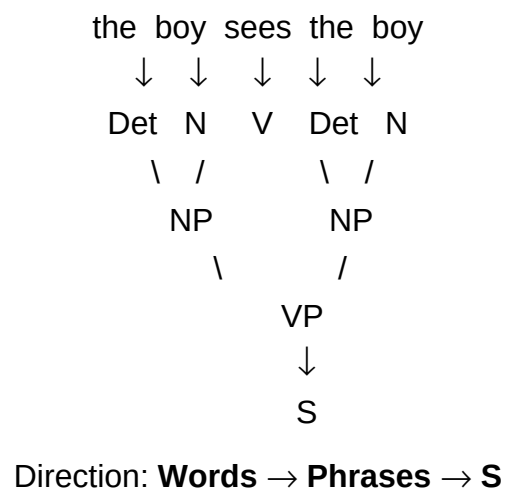
Feature	Top-Down	Bottom-Up
Begins with	Grammar	Input
Parse tree	Built from top	Built from bottom
Left recursion	Problematic	Can handle it
Unnecessary predictions	More likely	Less likely
Example approach	Recursive descent	Shift-reduce
Final result	Input sentence	Start symbol S

4. Simple Diagram

Top-Down



Bottom-Up



Cocke–Younger–Kasami (CYK) Algorithm

YK (Cocke–Younger–Kasami) is a parsing algorithm used to determine whether a given sentence/string can be generated by a **Context-Free Grammar (CFG)**.

Context Free Grammar:

We are given a [Context Free Grammar](#) $G = (V, X, R, S)$ and a string w , where:

- V is a finite set of variables or non-terminal symbols,
- X is a finite set of terminal symbols,
- R is a finite set of rules,
- S is the start symbol, a distinct element of V , and
- V and X are assumed to be disjoint sets.

The **CYK algorithm** checks whether a word fits the grammatical rules of a language using a **bottom-up dynamic programming table**.

Requirement: Chomsky Normal Form

Before applying CYK, the grammar should be converted into **CNF**.

The main CNF rules are:

- $A \rightarrow BC$

- $A \rightarrow a$

where:

- A, B, C are non-terminals
- a is a terminal

The CNF restriction allows CYK to divide every larger problem into **two smaller subproblems**.

Basic Idea

Suppose the input sentence is:

“a very heavy orange book”

There are 5 words:

Position	Word
1	a
2	very
3	heavy
4	orange
5	book

CYK creates a triangular $N \times N$ **table**.

Each cell $T[i,j]$ contains the non-terminals that can generate the substring from position i to j.

For example:

- $T[1,1] \rightarrow$ non-terminals that generate **“a”**

- $T[2,3] \rightarrow$ non-terminals that generate “**very heavy**”
- $T[1,5] \rightarrow$ non-terminals that generate the **entire sentence**

Important terms

Symbol	Meaning	Example
A	Adjective	heavy
AP	Adjective Phrase	very heavy
Nom	Nominal/Noun group	orange book
Det	Determiner	a
NP	Noun Phrase	a very heavy orange book

The CYK table is used to find **which grammar symbol(s) can generate each word or group of words.**

1. CYK Table

Start ↓ / End →	1 a	2 very	3 heavy	4 orange	5 book
1 a	T11	T12	T13	T14	T15
2 very		T22	T23	T24	T25
3 heavy			T33	T34	T35
4 orange				T44	T45
5 book					T55

What does T11 mean?

The notation **T_{ij}** is very simple:

- **i = starting word position**
- **j = ending word position**

For example:

T11

T11 = words from position 1 to position 1

That means only:

a

So we find which grammar symbol can generate “**a**”.

If the grammar has:

Det → a

then:

T11 = {Det}

T22

Position 2 to position 2:

very

If:

Adv $\rightarrow$ **very**

then:

T22 = {**Adv**}

T33

Position 3 to position 3:

heavy

If:

A $\rightarrow$ **heavy**

then:

T33 = {**A**}

If another rule also generates heavy, more than one symbol can appear.

T44

Position 4 to position 4:

orange

If the grammar contains:

A → **orange**

then:

T44 = {A}

If orange can also be a noun in the grammar, then Nom may also appear:

T44 = {A, Nom, AP}

This is called **ambiguity**.

T55

Position 5 to position 5:

book

If:

Nom → **book**

then:

T55 = {Nom}

3. What about T12?

Now we move beyond individual words.

T12 means:

Start at word 1 and end at word 2.

So:

T12 = “a very”

We check whether any grammar rule can generate “a very”.

If no rule can generate it, then:

T12 = $\emptyset$

where $\emptyset$ means empty/no matching grammar symbol.

4. What is T23?

T23 means:

Words 2 through 3

So:

T23 = “very heavy”

We already know:

very $\rightarrow$ Adv

heavy $\rightarrow$ A

If the grammar contains:

$AP \rightarrow Adv\ A$

then:

$Adv + A$

↓

AP

Therefore:

$T_{23} = \{AP\}$

5. What is T45?

T45 means:

Words 4 through 5

So:

$T_{45} = \text{“orange book”}$

If:

orange $\rightarrow A$

book $\rightarrow Nom$

and the grammar has:

$Nom \rightarrow A\ Nom$

then:

A + Nom

↓

Nom

Therefore:

T45 = {Nom}

6. What is T15?

This is the **most important cell**.

T15 means:

From word 1 to word 5

Therefore:

T15 = “a very heavy orange book”

This is the **complete sentence/phrase**.

The CYK algorithm checks whether the **start symbol** is present in T15.

If:

S ∈ T15

or, in this example, if the start symbol is **NP** and:

NP ∈ T15

then the input is **accepted**.

7. Complete idea

Think of the table as building the sentence **from small pieces to bigger pieces**:

Individual words



Two-word phrases



Three-word phrases



Four-word phrases



Complete phrase

For example:

heavy

↓
A

very + heavy

↓
AP

orange + book

↓
Nom

very heavy + orange book

↓
Nom

a + Nom

↓
NP

The important terms

Term	Meaning	Example
T11	Position 1 to 1	“a”

T22	Position 2 to 2	“very”
T33	Position 3 to 3	“heavy”
T44	Position 4 to 4	“orange”
T55	Position 5 to 5	“book”
T23	Position 2 to 3	“very heavy”
T45	Position 4 to 5	“orange book”
T15	Position 1 to 5	“a very heavy orange book”

★ **Easy rule to remember**

T_{ij} = substring starting at word i and ending at word j .

So:

T_{13} = “a very heavy”

T_{24} = “very heavy orange”

T_{35} = “heavy orange book”

T_{15} = the whole input: “a very heavy orange book”

That is the basic idea of the **CYK table**.